

Low Power System Design

Project Proposal - Group 4

Overview

The goal of the project is to design a low-power embedded system capable of measuring the temperature of an external thermistor in a water-bath. The system must operate from a cold boot and acquire a temperature reading every 30 seconds over a total duration of 5 minutes. The collected measurements then have to be transmitted to the project server via WiFi. The design must focus on minimizing energy consumption while maintaining stable and accurate temperature readings.

Problem Description

The challenge is to achieve both high measurement stability and the lowest possible energy consumption throughout the full measurement cycle. After each wake-up it should be managed the thermistor sampling sequence, buffer results in memory, and place the device back into deep sleep. The WiFi transmission is responsible for the vast majority of total power consumption, making timing, batching, and short on-air intervals critical for meeting the project's energy constraints.

System Overview

Hardware Design

Components

- **ESP32-C6 Devkit**
Provides integrated WiFi 6, an ultra-low-power deep sleep mode, and a 12-bit ADC for thermistor measurements.
- **Thermistor (TS-67 / 15K-1)**
A 15k Ω NTC thermistor used in a resistive voltage divider. Its temperature-dependent resistance allows precise analog measurement with minimal bias current.
- **Resistor**
The reference resistor forms a voltage divider with the thermistor to convert temperature-dependent resistance changes into a measurable voltage.

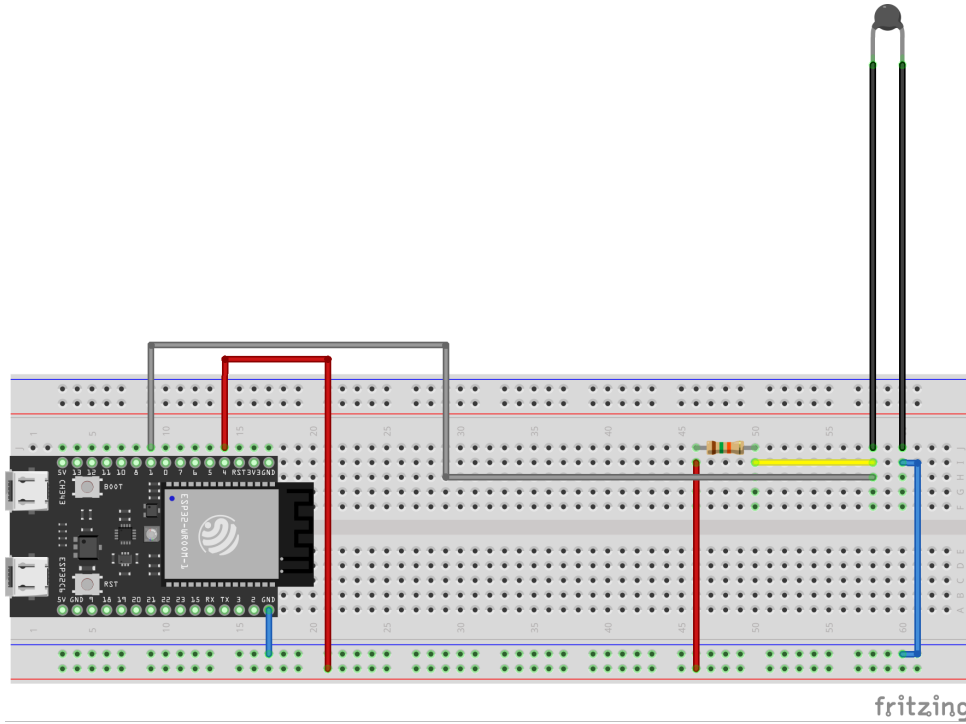


Figure 1: Proposed Circuit

Voltage divider

The following equation describes the output voltage of a resistive voltage divider used for thermistor-based temperature measurement. The thermistor with temperature-dependent resistance $R_{\text{therm}}(T)$ and the fixed reference resistor R_{ref} are connected in series between the supply voltage V_{in} and ground. The ADC measures the voltage at the junction of the two resistors. As the thermistor resistance changes with temperature, the ratio between $R_{\text{therm}}(T)$ and the total resistance determines the resulting output voltage V_{out} .

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_{\text{therm}}(T)}{R_{\text{therm}}(T) + R_{\text{ref}}}$$

The power consumption of the voltage divider depends on its total resistance. The current is

$$I = \frac{V_{\text{in}}}{R_{\text{therm}}(T) + R_{\text{ref}}}$$

and the power drawn from the supply is

$$P = \frac{V_{\text{in}}^2}{R_{\text{therm}}(T) + R_{\text{ref}}}.$$

A higher total resistance reduces the current and therefore lowers the power consumption, while a lower total resistance increases it.

Accuracy considerations. Besides power consumption, the choice of the divider resistance also affects the measurement accuracy. Very high resistor values increase the output impedance of the divider, which can lead to voltage errors due to the finite input impedance and sampling characteristics of the ADC. Large resistances additionally

increase thermal noise and make the measured signal more sensitive to external interference. Conversely, very low resistor values improve accuracy and noise robustness, but increase the static current draw. In practice, the total divider resistance must balance energy consumption and accuracy.

Software Design

Boot Sequence

Upon cold boot or wake-up from deep sleep, the microcontroller initializes essential peripherals: setup for the thermistor measurement, saving of data, and transimition over WiFi. A simple state-tracking mechanism stored in RTC memory determines whether the system is starting transmitting or only continuing the sampling cycle.

Measurement Logic

Every 30 seconds, the device wakes from deep sleep and performs a single temperature measurement. The thermistor bias is enabled for a short period, an ADC reading is taken and the equation is applied to convert the raw reading into a temperature value. The sample is appended to a buffer along with a timestamp. After the measurement step, the system immediately returns to deep sleep to minimize CPU active time.

Data Transmission

After ten measurements have been collected over the 5-minute period, the device performs a wake-up and activates the WiFi transceiver. It connects to the project server and uploads all buffered measurement values. Once the upload has been done, the WiFi is disabled and the system transitions back to deep sleep to continue a new cycle if needed.

Error Handling

Potential faults include invalid ADC readings, or WiFi connection failures. The firmware retries ADC measurements once before discarding a faulty sample. In the event of network errors, the device uses a short backoff strategy and attempts retransmit in the next cycle (after again 5min), with old and new data.

Power Optimization

- **WiFi Transceiver Activity**

By far the largest energy cost. Transmitting all data at once at the end of the measurement period minimizes radio-on time and reduces peak current draw.

- **CPU Active Time**

Keep operations as short as possible to reduce active cpu time.

- **Measurements**

Very low energy usage. Switch on the power of voltage divider while deep-sleep and choose the best resistor by optimizing energy consumption and accuracy.

- **Error Handling**

In case of an error, try to keep all above optimisation stable.